

HPF-Pre Series 3 response curves

April 29, 2026

1 Response curves of “HPF-Pre Series 3” high pass filter preamp

This is a demo of my audio testing library, using my HPF-Pre Series 3 preamp as device-under-test. For this test, I do not have any speakers hooked up. I’ve got a cheap USB audio adapter.

Hardware setup: For now, I’ve connected a patch cord from Line Out to Line In on my audio adapter.

```
[1]: %load_ext autoreload
      %autoreload 2
      import numpy as np
      import matplotlib.pyplot as plt
      import time
      import pyAudioTest
```

sine frequency 164.190673828125

2 Checking out the audio test setup

This is the guts of the test program that I wrote for my pyAudioTest library. It makes sense to run this and make sure that the audio subsystem is working before trying to test anything with it.

```
[18]: '''
      Make sure there's no audio already running
      '''

      if "audio" in locals():
          pyAudioTest.stopAudio(stream)
          del audio, stream

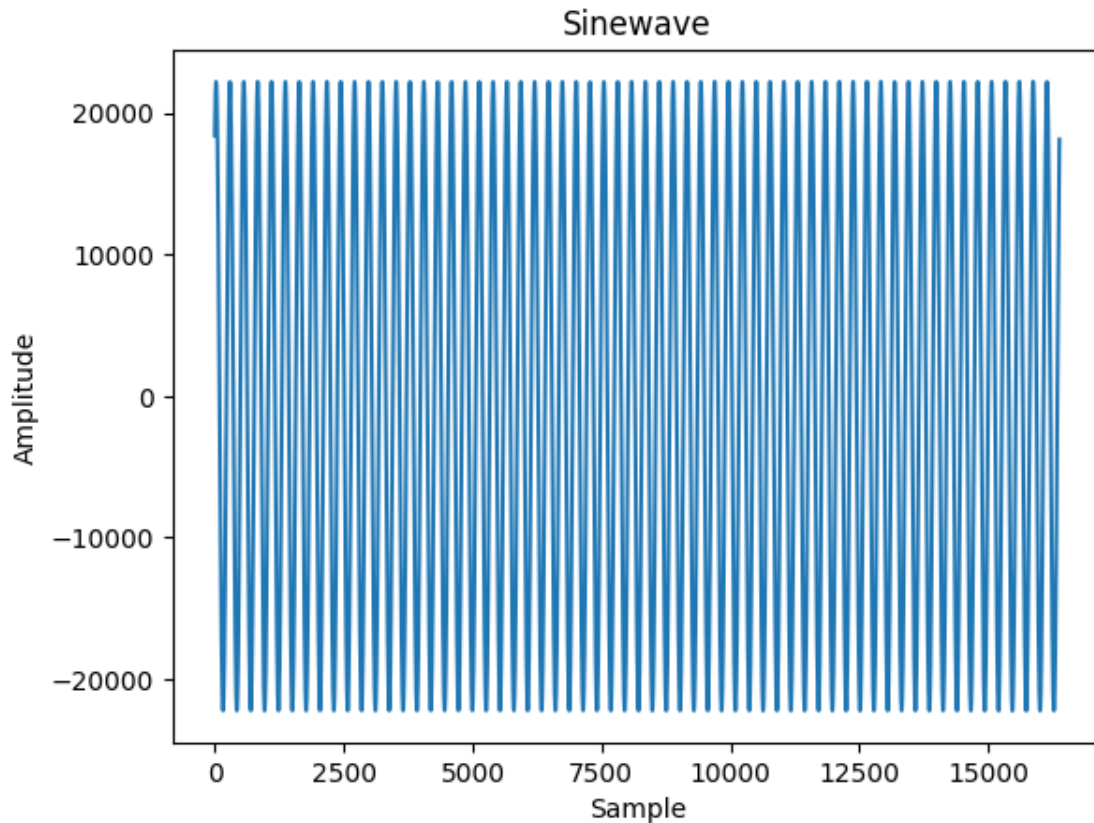
      audio, stream = pyAudioTest.startAudio()
```

Check to make sure that the test setup generates a sensible signal. Repeat the sinewave test until the amplitude is below 32000.

```
[28]: pyAudioTest.globals['generate'] = 2 # Choose sinewave output for now
      time.sleep(2)
      pyAudioTest.waitCycle()
      data = pyAudioTest.analyzeAudio(stream)
```

```
plt.plot(data['data'][0])
plt.title('Sinewave')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
```

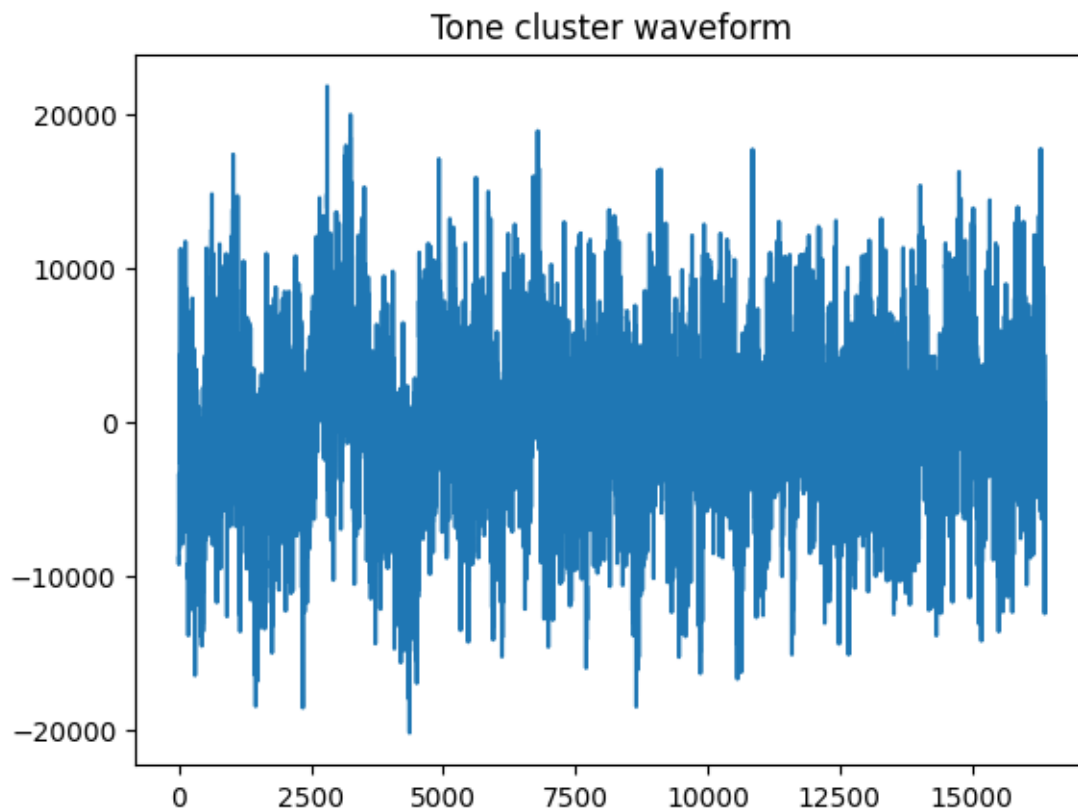
```
[28]: Text(0, 0.5, 'Amplitude')
```



Now switch to the tone cluster waveform. This is what we'll be using for the frequency response test. Again, adjust the volume control until the overall amplitude is below 32000 units.

```
[29]: pyAudioTest.globals['generate'] = 1 # the tone cluster waveform
time.sleep(2)
pyAudioTest.waitForCycle()
data = pyAudioTest.analyzeAudio(stream)
plt.plot(data['data'][0])
plt.title('Tone cluster waveform')
```

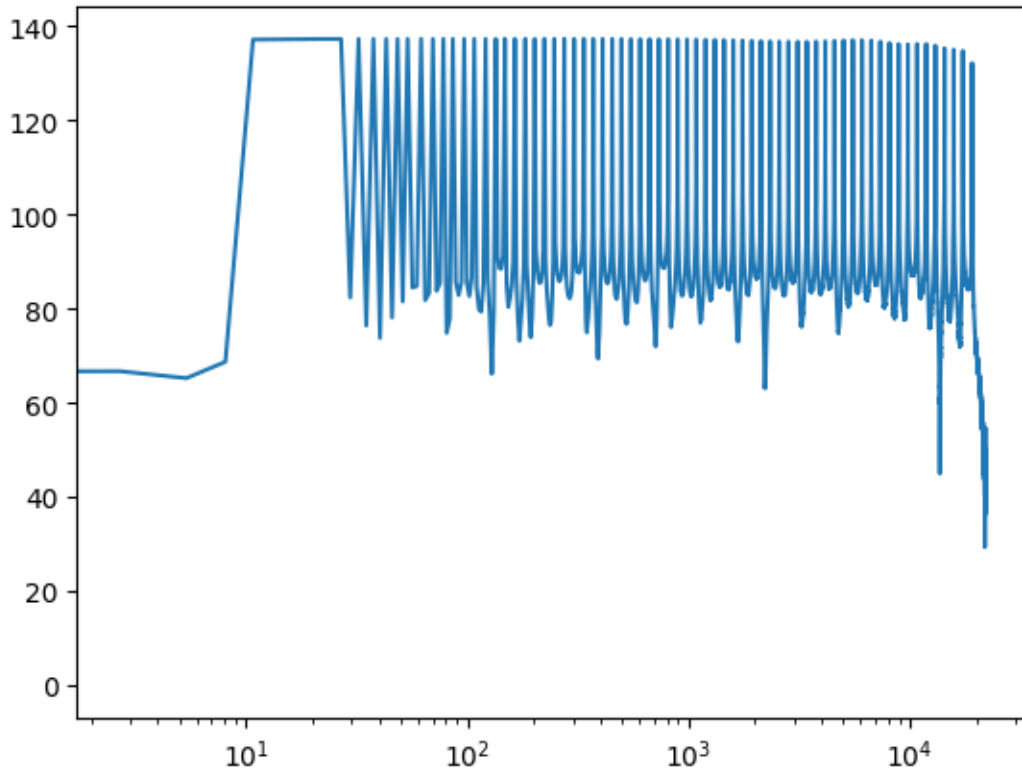
```
[29]: Text(0.5, 1.0, 'Tone cluster waveform')
```



Before we continue, let's see what this waveform actually produces in frequency space. It's a bunch of discrete peaks, all of which are 1 unit wide. In subsequent graphs, I will only show the tips of the peaks. The rest is noise.

```
[30]: plt.semilogx(pyAudioTest.frequencyScale, data['dbfft'][0])
```

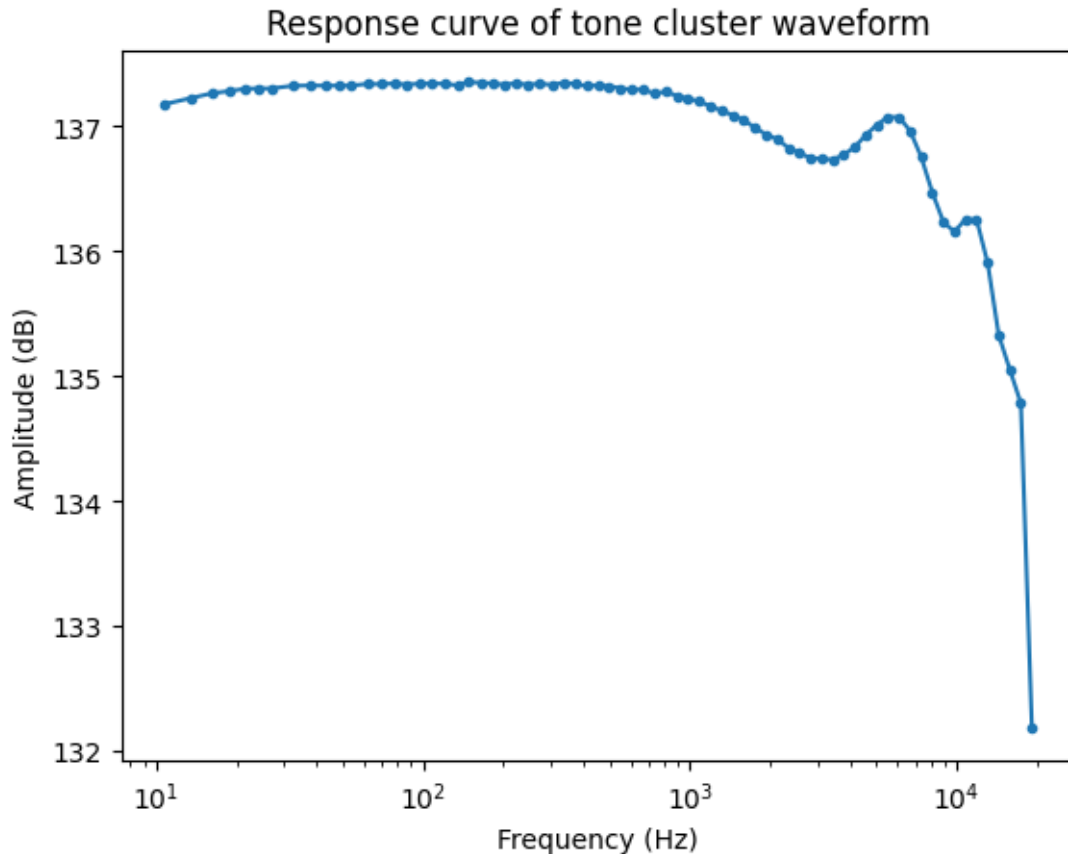
```
[30]: [<matplotlib.lines.Line2D at 0x187ec622270>]
```



Here's a plot of just the tones in the tone cluster waveform. Some things are immediately noticeable. First, the response curve looks quite smooth. That's good. It means the analyzer produces a good display from a single block of data with no averaging needed. Second, the curve looks wiggly at high frequencies, but the wiggles are all within 1/2 dB of a response curve that you'd expect if the 3 dB point is at 20 kHz. The audio adapter is performing well.

```
[32]: plt.semilogx(pyAudioTest.toneFrequency, data['dbfft'][0][pyAudioTest.
    ↪toneIndex], marker = '.')
plt.title('Response curve of tone cluster waveform')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (dB)')
```

```
[32]: Text(0, 0.5, 'Amplitude (dB)')
```



3 On with the testing

Now I've changed my test setup. I ran patch cables from Line Out of the audio adapter to the input of the HPF-Pre, and from the output of the HPF-Pre to Line In. The HPF has only one knob, for the cutoff frequency of the filter. I will try 3 knob settings.

First I will measure and record a “baseline” which is the response of the patch cord. This defines a 0 dB level at each frequency.

```
[33]: pyAudioTest.waitCycle() # make sure one "cycle" of data have been collected
      baseline = pyAudioTest.analyzeAudio(stream)['dbfft']
```

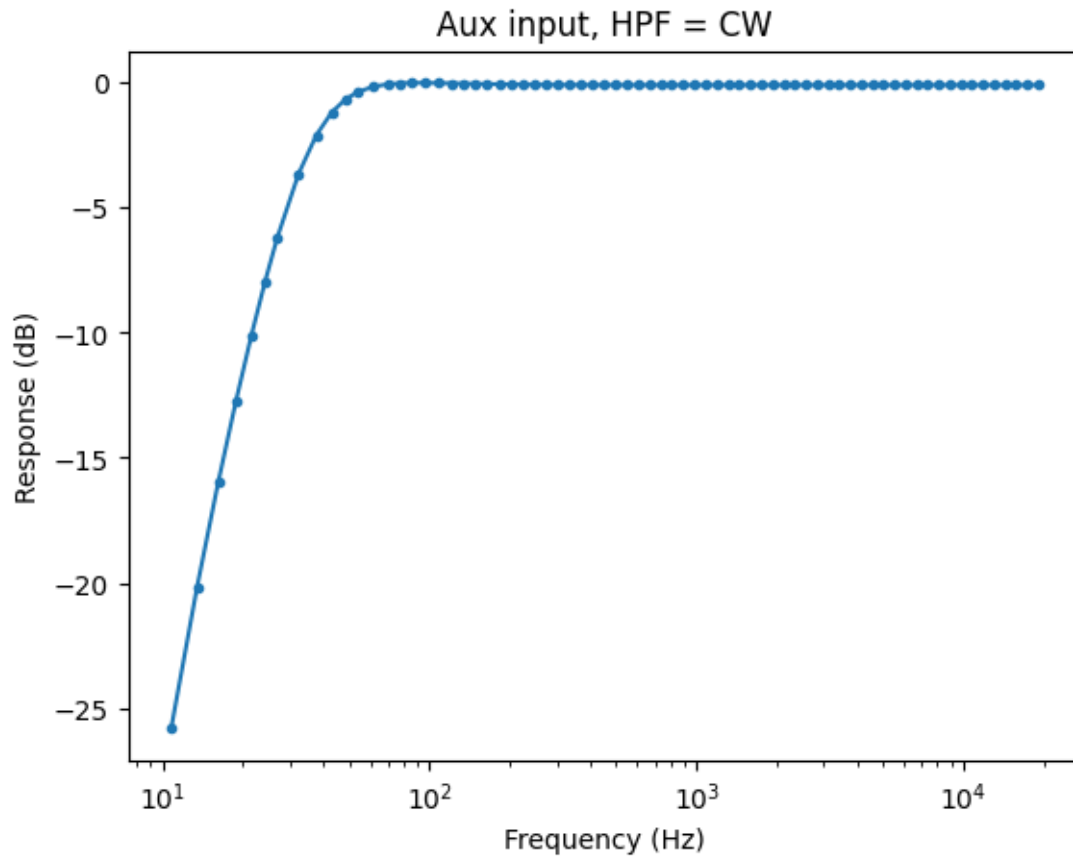
Next, I will define a function for the response curve, for convenience,

```
[38]: def response():
      pyAudioTest.waitCycle() # make sure one "cycle" of data have been collected
      data = pyAudioTest.analyzeAudio(stream)
      return (data['dbfft'] - baseline)[0][pyAudioTest.toneIndex]

in_1 = response()
```

```
[39]: plt.semilogx(pyAudioTest.toneFrequency, in_1, marker = '.')
plt.title('Aux input, HPF = CW')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Response (dB)')
```

```
[39]: Text(0, 0.5, 'Response (dB)')
```



Now I'm going to record curves at 3 positions of the frequency knob.

```
[52]: in_cw = response()
```

```
[53]: in_center = response()
```

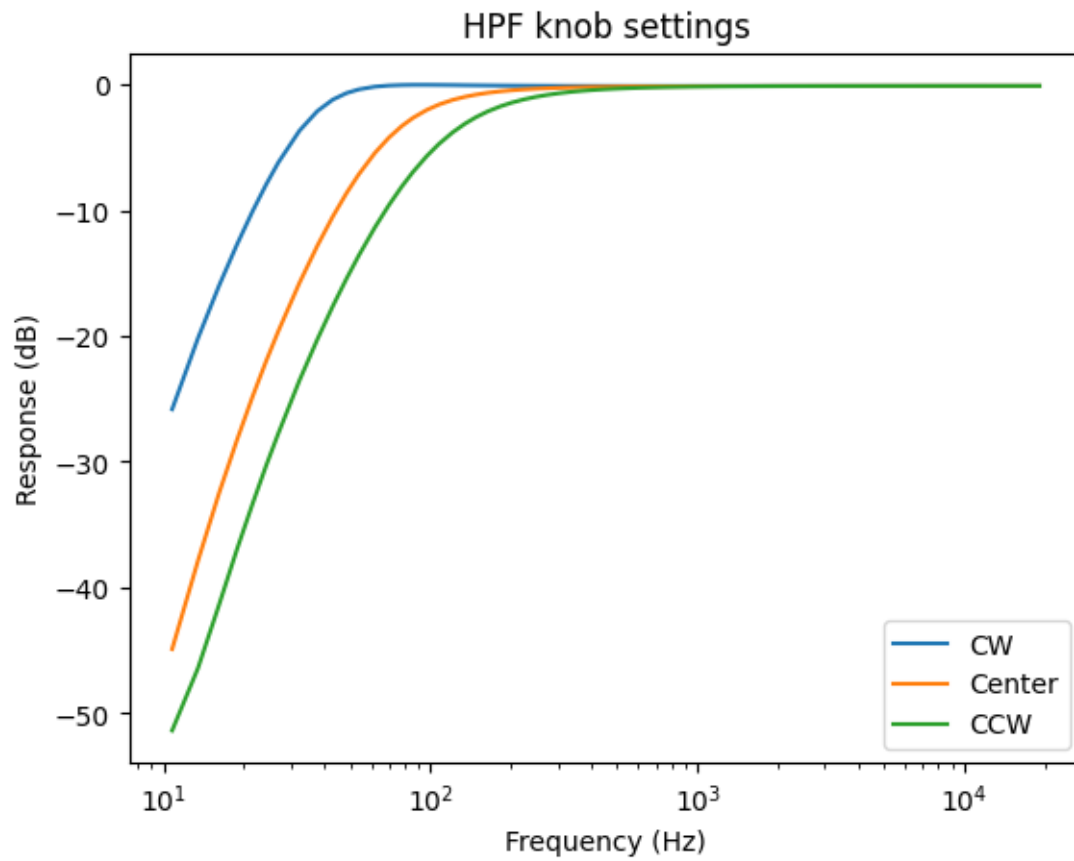
```
[54]: in_ccw = response()
```

Finally, I'll graph all of the curves together, to see how the frequency knob affects the response curve of the HPF.

```
[55]: plt.semilogx(pyAudioTest.toneFrequency, in_cw, label = 'CW')
plt.semilogx(pyAudioTest.toneFrequency, in_center, label = 'Center')
plt.semilogx(pyAudioTest.toneFrequency, in_ccw, label = 'CCW')
```

```
plt.title('HPF knob settings')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Response (dB)')
plt.legend()
```

[55]: <matplotlib.legend.Legend at 0x187f1d73dd0>



[]: